

Assignment 3 Bridge Pattern

Identification

Assignment title: Bridge Pattern

Full name: Yermekkyzy Assel

Group: SE-2524

Topic: Bridge Pattern - Notifications

GitHub repository link: <https://github.com/yerassell/assignment3-bridge-pattern.git>

Base commit: b0141ba3923d26aac2c2f25ac332d71c43cdb0a5

Last commit: 62113016c724c47b20fd36239a1546d7ce15bb1b (before adding report and adding commit to readme)

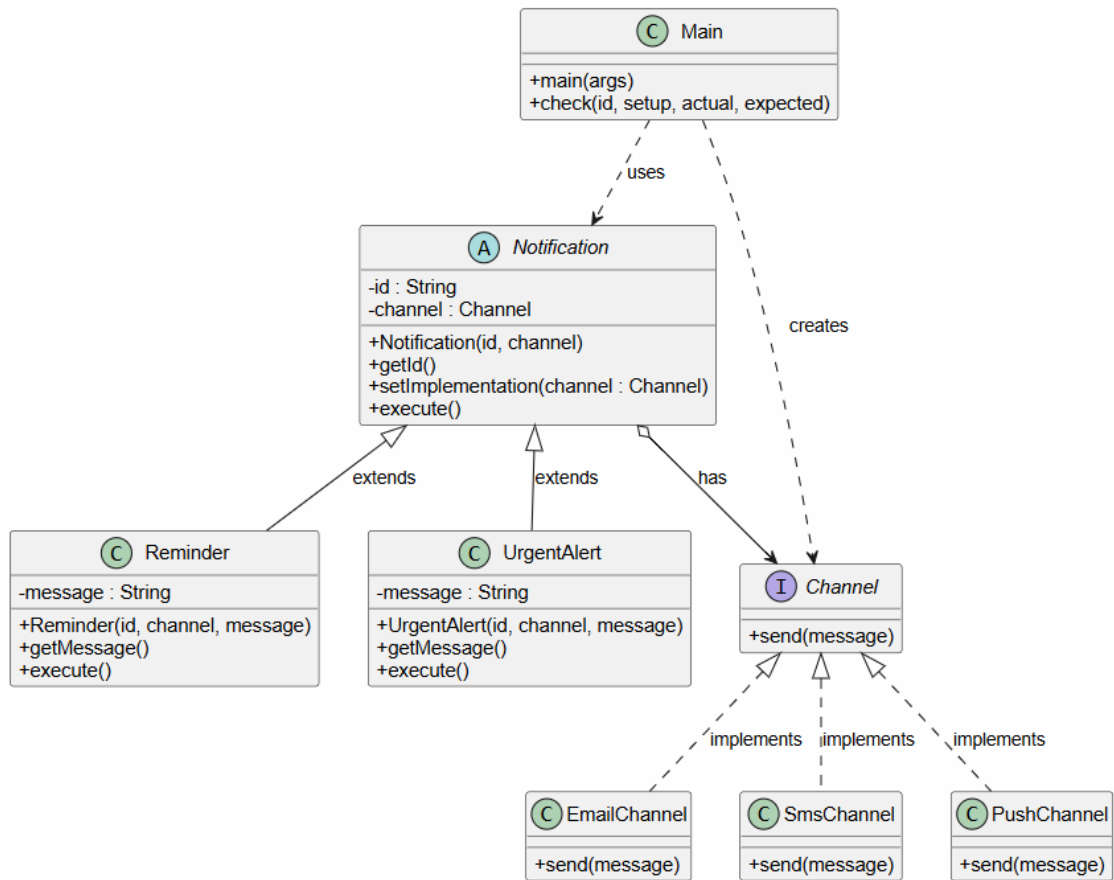
Introduction

Two independent dimensions and why bridge fits:

This Java application uses Bridge to connect two independently varying class hierarchies: Notification and Channel. Notifications abstraction is what kind of notifications are being sent, like Reminder or UrgentAlert. Channel implementor is how those notifications are being delivered, like through EmailChannel, SmsChannel and PushChannel.

Without Bridge pattern, there would be created $2 \times 3 = 6$ classes. But adding one more channel or notification type would require additional classes and lead to class explosion which is a bit dirty. The Bridge Pattern avoids this problem by separating two dimensions and connecting them by composition. Adding new channel will not change notifications, while adding new notification type will not change channel types.

UML class diagram



Picture 1. UML class diagram

Clean Code

1. Separation of responsibilities

Explanation:

The Channel interface has one responsibility which is to define how a message is sent. It does not know about Notification class and is not affected by external changes.

Excerpt (Channel.java):

```
public interface Channel {
    String send(String message);
}
```

2. Meaningful names

Explanation:

The class and method names are self-explanatory. The user understands what they do without needing comments.

Excerpt (Notification.java):

```
public abstract class Notification {
    private final String id;
    protected Channel channel;
```

```
protected Notification(String id, Channel channel){
    this.id=id;
    this.channel=channel;
}
```

3. *Small focused methods*

Explanation:

The method execute() does one thing only which is to build a message and delegate it to the channel. It does not do anything else.

Excerpt (Line 14 in Reminder.java):

```
public String execute(){
    return channel.send("Reminder: " + message);
}
```

4. *Avoidance of duplication*

Explanation:

When checking from T1 to T7 except for T5, they all use same logic. Instead of repeating them for each check, commonly used comparison is written in check() that avoids duplication and leads to short and clean code.

Excerpt (Line 51 in Main.java):

```
private static int check(String id, String setup, String actual, String
expected){
    if (actual.equals(expected)){
        System.out.println(id + " PASS | " + setup + " | result=" + actual);
        return 1;
    }
    else{
        System.out.println(id + " FAIL | " + setup + " | result=" + actual);
        System.out.println("Expected: " + expected);
        return 0;
    }
}
```

5. *Encapsulation of state*

Explanation:

The id field is private final so it cannot be changed after construction. It also cannot be accessed from outside the class, but can only be read by getID(). This encapsulation protects the abstraction class even for T5 where the implementor was changed.

Excerpt (line 4 in Notification.java):

```
public abstract class Notification {
    private final String id;
    protected Channel channel;

    protected Notification(String id, Channel channel){
        this.id=id;
        this.channel=channel;
    }
}
```

Conclusion

How I3 was added:

At first the I3, Push Channel, was added as a new class inside the implementor package. In Main.java, I added T6 and T7 using the same construction logic as for previous T1-T4. In Main.java I also changed one line to show that 7/7 checks passed instead of initial 5/5. The rest of the code, Notification.java, Reminder.java, UrgentAlert.java were not changed. This proves that in Bridge pattern the changes in implementor does not affect the abstraction. Channel.java and EmailChannel.java with SmsChannel.java were not changed as well. The changes after Base commit are shown in extension.diff.

Trade-off of Bridge:

Using the Bridge pattern leads to separation of logic across two dimensions that can make it harder to trace how things work. To understand the flow of, for example, how execute() works in Reminder, first we see that it calls channel.send(), but to see which channel, we go to Main where the object is created. It is not useful for a small system with only one implementation, but for our application with two independent dimensions it is flexible.

Bridge vs. Adapter:

Bridge is used to separate Abstraction and Implementor, two independent dimensions, from the beginning. These two hierarchies are connected by composition. The changes in abstraction does not affect implementor and vice versa.

Adapter is used when an existing class has and interface that is incompatible for what the client expects. Instead of rewriting the existing old system, adapter implements target interface and wraps old system to produce the output the client wants.

Sources used:

- Lecture 4
- <https://refactoring.guru/design-patterns/bridge/java/example>
- PlantUML